

BÁO CÁO ĐÁNH GIÁ KIẾN TRÚC BACKEND & LỘ TRÌNH TỐI ƯU PRODUCTION-GRADE

Dự án: Akira LaDux (Spring Boot + PostgreSQL + Redis)

Người nhận: Khang (Lead Developer) | Phân tích bởi: Tui (AI Assistant) | Phương pháp: Nguyên lý 80/20 (Ưu tiên giải quyết nút thắt cổ chai) | Ngày lập: 23/07/2026

1. Đánh Giá Tổng Quan & Điểm Mạnh Đã Đạt Được

Chào Khang! Hệ thống Backend hiện tại của Khang đã vượt mức MVP thông thường và có nền móng kiến trúc rất tốt. Việc lựa chọn **Modular Monolith** ở giai đoạn này là cực kỳ chuẩn xác, tránh được chi phí vận hành phức tạp không cần thiết của Microservices khi chưa đạt scale lớn.

Tóm tắt điểm tích cực nổi bật:

- Kiểm soát Transaction chặt chẽ:** Đã tắt `spring.jpa.open-in-view=false` giúp tránh tình trạng giữ Database Connection quá lâu trong suốt lifecycle của HTTP Request.
- Chống Oversell tin cậy:** Dùng Atomic Update trực tiếp dưới DB (SQL Update với điều kiện kho) tại `ProductRepository.java:59`, giúp bảo vệ tính nhất quán dữ liệu khi có concurrency cao.
- Luồng Tiền/Kho thiết kế bài bản:** Order & Payment có State Machine rõ ràng, Webhook có Idempotency chống duplicate payment, tích hợp Stock Ledger và cơ chế Rollback tồn kho/coupon khi thanh toán thất bại.
- Nền tảng Monitoring & Testing:** Đã tích hợp Spring Boot Actuator cho healthcheck/metrics và sử dụng Testcontainers cho Integration Testing.

2. Phân Tích 80/20: 7 Điểm Nghẽn & Rủi Ro Cần Khắc Phục

Áp dụng nguyên tắc 80/20, mặc dù nền tảng tốt nhưng backend **chưa đạt chuẩn Production-Grade** do có 2 lỗi về correctness/hiệu năng rất nghiêm trọng và một số rủi ro an toàn vận hành:

STT	Hạng Mục Lỗi	Mô Tả & Vị Trí Code	Mức Độ Ảnh Hưởng
1	Test Suite Đỏ (Fail 2/34)	<code>LoginRateLimitTest.java:18</code>: Test expect capacity = 5 nhưng config <code>application.properties:59</code> cài 10. <code>SeedDataPersistenceTest.java:41</code>: Test expect 10 category nhưng DB seed thực tế chỉ có 6.	P0 - NGHIÊM TRỌNG Làm hỏng CI/CD pipeline, mất niềm tin vào tự động hóa test.
2	Pagination In-Memory (Hibernate)	<code>OrderRepository.java:20</code> : Query phân trang <code>Page<Order></code> đồng thời JOIN FETCH collection <code>payments</code> . Cảnh báo Hibernate: <i>firstResult/maxResults specified with collection fetch; applying in memory.</i>	P0 - NGUY HIỂM Hibernate phải load TOÀN BỘ bảng Order vào RAM rồi mới phân trang trong Java. Dễ gây Out Of Memory (OOM) khi dữ liệu lớn.

STT	Hạng Mục Lỗi	Mô Tả & Vị Trí Code	Mức Độ Ảnh Hưởng
3	Cache Invalidation "Xóa Cỏ Làng"	<code>OrderServiceImpl.java:113</code> : Khi Checkout, dùng <code>allEntries=true</code> xóa sạch cache của orders, orderItems, histories, products, coupons, carts.	P1 - THẤP HIỆU NĂNG Tỷ lệ Hit-rate của Redis cực kỳ thấp, Redis trở thành chi phí thừa thay vì tăng tốc.
4	API Contract Trả Trực Tiếp <code>Page<T></code>	<code>ProductController.java:18</code> : API trả về trực tiếp đối tượng <code>org.springframework.data.domain.Page</code> .	P0 - BẮT MỜI API Cấu trúc JSON của Spring Page không ổn định giữa các phiên bản Spring Boot, gây vỡ contract với Frontend/Mobile.
5	Security REST Chưa Hoàn Chỉnh	<code>SecurityConfig.java:110</code> : Mới custom <code>AccessDeniedHandler</code> (403), chưa có <code>AuthenticationEntryPoint</code> trả về JSON chuẩn cho request 401 unauthenticated.	P0 - SECURITY Request chưa đăng nhập nhận về HTML 401 mặc định hoặc redirect thay vì response JSON nhất quán.
6	Flyway Dev Rủi Ro Cực Cao	<code>application-dev.properties:10</code> : Cấu hình <code>clean-on-validation-error=true</code> .	P0 - MẤT DỮ LIỆU Khi migration dev bị lệch checksum, Flyway sẽ tự động WIPE (xóa sạch) Database của môi trường dev.
7	Hard Delete Thực Thể Nhạy Cảm	<code>UserServiceImpl.java:228</code> : Đang dùng Hard Delete xóa hẳn record User.	P0 - SAI NGHIỆP VỤ Làm vỡ khóa ngoại (Foreign Key) với các đơn hàng/thanh toán cũ của User đó. Cần đổi sang Soft Delete / Deactivate.

3. Lộ Trình Nâng Cấp Tối Ưu (Prioritized Roadmap)

GIAI ĐOẠN P0 - Sửa Lỗi Đáng Dấn & Ổn Định Tức Thì (Làm Ngay)

- **Fix Test Suite:** Đồng bộ config RateLimit (đưa capacity test/config về cùng giá trị 10 hoặc 5) và bổ sung Seed Data cho đủ 10 categories để CI xanh 100%.
- **Sửa Pagination Query:** Tách làm 2-step query (Query lấy Page ID trước, sau đó query fetch detail theo ID) HOẶC dùng DTO Projections cho danh sách phân trang.
- **Bổ Sung `AuthenticationEntryPoint`** : Viết custom JSON response (HTTP status 401, error code, message) trong Spring Security.
- **Tắt Flyway Auto Clean:** Đổi `flyway.clean-on-validation-error=false`. Chỉ bật tính năng clean ở Maven profile riêng biệt (`local-destructive`).
- **Chuyển sang Soft Delete:** Thêm field `is_deleted` / `status` cho User, Product, Supplier để tránh xóa vĩnh viễn record.

- **Chuẩn hóa `PageResponse<T>`** : Tạo DTO wrapper chuẩn gồm `content` , `pageNo` , `pageSize` , `totalElements` , `totalPages` , `isLast` .

GIAI ĐOẠN P1 - Nâng Cấp Hiệu Năng & Giám Sát (Production Readiness)

- **Thiết kế lại Redis Cache**: Chỉ cache Hot-Read (Danh mục, Sản phẩm nổi bật). Evict cụ thể theo Key/ID (`@CacheEvict(key = "#id")`) thay vì ``allEntries=true``. Đặt TTL phù hợp cho từng loại cache. Không cache dữ liệu biến động cao như Order/Payment.
- **Cấu hình Observability**: Thêm dependency `micrometer-registry-prometheus` vào `pom.xml` để Prometheus cào được metrics từ Actuator.
- **Structured Logging & Tracing**: Thêm Logback JSON layout + Filter sinh `X-Request-Id` (MDC Logging) để trace toàn bộ flow từ Controller tới Service.
- **Database Tuning**: Bổ sung Dashboard theo dõi P95/P99 latency, HikariCP pool connection, Redis latency. Tune HikariCP max-pool-size phù hợp.
- **DTO Projections cho Admin List**: Tránh load toàn bộ Hibernate Entity Graph nặng nề khi làm báo cáo/danh sách Admin.

GIAI ĐOẠN P2 - Tối Ưu Kiến Trúc Tương Lai (High Scalability)

- **Tách Bounded Context Package**: Tổ chức code lại theo domain: `identity` , `catalog` , `ordering` , `payment` , `inventory` , `procurement` , `crm` .
- **Domain Events + Transactional Outbox Pattern**: Tách các tác vụ phụ thuộc (Gửi email, tính điểm thưởng, gửi notification) ra khỏi Transaction chính của Checkout thông qua Outbox Table + Event Publisher.
- **Idempotency Key**: Áp dụng Idempotency Header cho `POST /api/v1/orders` để ngăn chặn người dùng bấm submit trùng lặp khi mạng lag.
- **Concurrency Load Testing**: Viết JMeter / Gatling script test đồng thời checkout cùng 1 sản phẩm còn 1 tồn kho, đồng thời redeem coupon.

4. Giải Pháp Kỹ Thuật Chi Tiết Cho Các Vấn Đề Trọng Yếu

4.1. Giải quyết triệt để Pagination In-Memory (2-Step Query Pattern)

Thay vì fetch JOIN collection trong Query có phân trang, Khang nên dùng kỹ thuật 2 bước dưới đây:

```
// 1. Query lấy danh sách ID đã được phân trang chuẩn dưới DB (Không fetch collection)
@Query("SELECT o.id FROM Order o WHERE o.user.id = :userId")
Page<Long> findOrderIdsByUserId(@Param("userId") Long userId, Pageable pageable);

// 2. Fetch đầy đủ chi tiết Entity/Collection theo danh sách IDs vừa lấy
@Query("SELECT DISTINCT o FROM Order o LEFT JOIN FETCH o.payments WHERE o.id IN :ids")
List<Order> findOrdersWithPaymentsByIds(@Param("ids") List<Long> ids);

// 3. Service Layer phối hợp:
public PageResponse<OrderDTO> getUserOrders(Long userId, Pageable pageable) {
    Page<Long> idPage = orderRepository.findOrderIdsByUserId(userId, pageable);
    if (idPage.isEmpty()) {
        return PageResponse.empty();
    }
    List<Order> orders = orderRepository.findOrdersWithPaymentsByIds(idPage.getContent());
    List<OrderDTO> dtos = orders.stream().map(orderMapper::toDto).toList();
    return PageResponse.of(dtos, idPage);
}
```

4.2. Standardized PageResponse DTO Wrapper

Tạo wrapper class cố định contract JSON giúp Frontend dễ tích hợp và không lo breaking change khi upgrade Spring Boot:

```
public record PageResponse<T>(
    List<T> content,
    int pageNo,
    int pageSize,
    long totalElements,
    int totalPages,
    boolean last
) {
    public static <T> PageResponse<T> from(Page<T> page) {
        return new PageResponse<>(
            page.getContent(),
            page.getNumber(),
            page.getSize(),
            page.getTotalElements(),
            page.getTotalPages(),
            page.isLast()
        );
    }
}
```

4.3. Custom REST AuthenticationEntryPoint (HTTP 401 JSON)

Đảm bảo mọi request chưa xác thực vào `/api/**` đều nhận về JSON thống nhất:

```
@Component
public class RestAuthenticationEntryPoint implements AuthenticationEntryPoint {
    @Override
    public void commence(HttpServletRequest request, HttpServletResponse response,
        AuthenticationException authException) throws IOException {
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);
        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);

        Map<String, Object> body = Map.of(
            "timestamp", Instant.now().toString(),
            "status", 401,
            "error", "Unauthorized",
            "message", "Yêu cầu xác thực không hợp lệ hoặc token đã hết hạn.",
            "path", request.getServletPath()
        );
        new ObjectMapper().writeValue(response.getOutputStream(), body);
    }
}
```

Tóm lại từ Tui gửi Khang:

Khang đã xây dựng một bộ khung backend rất vững chắc. Việc cần làm ngay tuần này là **đưa Test Suite về màu xanh (Green CI)**, **sửa Query phân trang Order** để tránh treo RAM RAM server, và **bảo vệ DB dev trước Flyway clean**. Làm xong P0 là hệ thống đã đủ tự tin để chạy Production an toàn!